

Beyond MCPs

Self-healing, verified GUI automation for accessible AI assistants

M1 REPORT: MVP AND PROPOSAL

CSE 493 – Final Year Project, Term 1 Software track

Authors:

Sharique Baig (28369)
Maira Aijaz (28552)
Barkhi Khan (29261)

Advisors:

Dr. Faisal Iradat
Pete Herzog (ISECOM)

October 29, 2026

Department of Computer Science,
School of Mathematics and Computer Science,
Institute of Business Administration, Karachi

Contents

1 Context **2**

2 Track and milestone **3**

3 Approach **3**

4 The demonstrator **4**

5 Evaluation **4**

6 Limitations **7**

7 Team contribution **7**

8 Plan for M2 to M5 **8**

9 AI Tools Used **8**

1 Context

Assistants are increasingly asked to operate software through its screen, because many everyday apps expose no API. Where an API or an MCP server exists it should be used; this project is about the apps that offer nothing else. Recorded scripts are the usual tool, and they fail in two ways: one renamed id sends a click nowhere, and a script that never checks the next screen cannot tell that the goal was not reached. For a blind or motor-impaired user, being told "please do this step yourself" is not a fallback, so the assistant must be right or say clearly that it is not.

The information a screen reader uses is usually present. On stock Android Settings, 78% of controls look nameless by their own attributes, but only 2% are nameless once the labels inside a clickable row are counted (measured with `path-b/hierdump.py`).

78% → 2%	**5 of 8**	**7 of 8**	**2 → 10/10**
nameless controls in Android Settings: by their own attributes, then counting row labels	single UI changes break a recorded script	are survived by the rules ladder, which stops honestly on the rest	French Settings intents: rules alone, then with the checked model

Figure 1: At a glance. Every figure is read from the published logs or the coverage measurement.

Platforms. Figure 2 is the project poster. It sorts platforms into agent-native (APIs and MCP: out of scope), legacy local apps without an API (our target class) and agent-hostile platforms. We evaluated a mock Instagram-style web app and stock Android Settings; Daraz, OLX and Instagram were *not* tested.

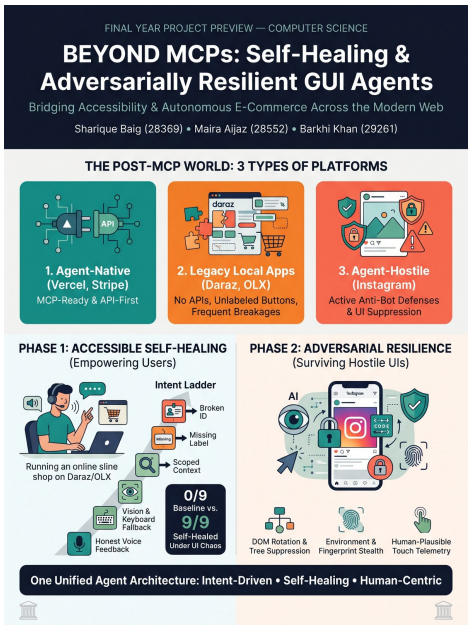


Figure 2: The project poster (earlier wording). Its Phase 2 claims are narrowed by the scope below.

Collaboration. The advisor is Dr. Faisal Iradat. The work started from a scoping brief written by a *synthie* (a long-running agent with its own host, identity and owner) about the failures of its own Android automation, and is carried out with Pete Herzog of the Institute for Security

and Open Methodologies (ISECOM), an open security research community that published the OSSTMM in 2001.

Scope. Surviving anti-bot detection is out of scope: we measure how a protection layer treats an agent and fail honestly, but build no evasion. A model is never trusted with an action (Section 3).

2 Track and milestone

The project is on the software track. This report is the M1 checkpoint: a working demonstrator of the core idea and a written proposal.

M1 requirement	Evidence
Working demonstrator of the core part	A nine-step flow runs end to end with next-screen checks; an Android layer runs on an emulator.
External integration works	Playwright drives a browser; `adb` drives an emulator; a language model is called through OpenRouter.
Written proposal	This report, including the plan in Section 8.
AI-tool disclosure	Section 9.

3 Approach

The tool follows three rules. **Intent, not locator:** the assistant states a goal such as "open the create-post flow", never a button. **Verify on the next screen:** a step counts only if a predicate on the next screen shows the goal was reached. **Stop honestly:** if nothing works, the tool says which step failed and why, ending "You may need to do this step yourself."

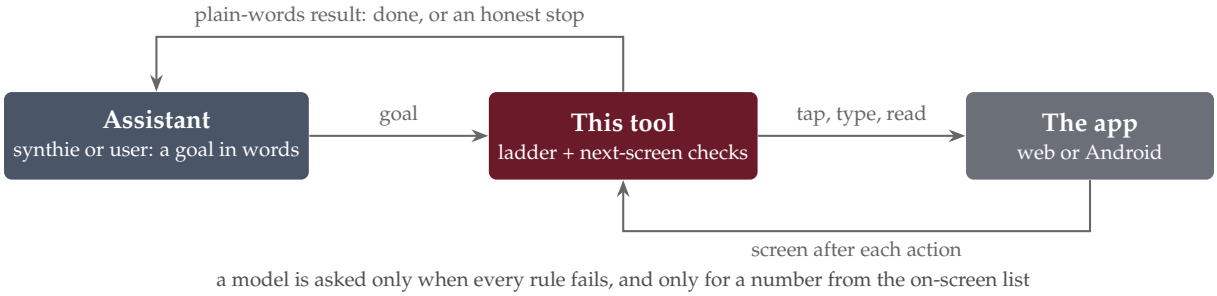


Figure 3: How the tool sits between an assistant and an app.

For each step it tries a ladder of ways to find the control (Figure 4). The model is asked only when every rule route fails; it may answer only with indices into the on-screen control list, its answer is a candidate judged by the same next-screen check, and an irreversible step (publishing) needs confidence of at least 0.8 and only the first choice. A model answer that passed its check is remembered so the model need not be asked again. The Android layer applies the same ladder to UiAutomator trees and pages a scrollable list when nothing has been tapped and a control may be below the fold. The tool can also say its result aloud, reporting success only when every check passed.



Each route counts only if it points at **exactly one** control. After every tap the **next screen is checked**: goal reached → done; goal not reached → next route; none works → **honest stop**.

Figure 4: The ladder of ways to find a control, tried left to right.

4 The demonstrator

- `node agent/fyp.mjs run` performs the nine-step create-post flow on the mock app, verified in the feed; flags select UI changes, the recorded-script arm, the model rung and voice.
- `node agent/fyp.mjs experiment` runs arms against UI changes and writes raw logs; reports and figures are built from those logs.
- `python path-b/dashboard.py` is a live Android view.

A regression set (unit suites, Android self-tests and three end-to-end runs) is run after every change to the harness. It needs only Node 18; there is no GPU requirement.

5 Evaluation

Method. One nine-step flow on the mock app; a flow *completes* only if every step passes its next-screen check. Eleven controlled UI changes (B1–B11: ids renamed, labels stripped, layout reversed, control duplicated, nested in a scroller, bare icon, overlay, hidden, reworded, relocated, planted instructions) are applied alone and together. Arms: a recorded script, the rules ladder, a bare model acting on its first pick, a model with a check but no rules, and the full system. Each cell is 10 runs with 95% Wilson intervals [1]. The model is qwen3.7-flash at temperature 0 with cached answers, so repeats show variation in the app and harness, not in the model. A run in which any model call failed is invalid and never quoted. Every number below is read from the published logs by a script.

Result 1: surviving UI change (Figure 5). On the clean app both arms complete 10/10, so the harness itself is sound. The recorded script drops to 0/10 under B1 (ids renamed), B2 (labels stripped), B6 (bare icon), B7 (overlay) and B8 (control hidden), that is five of the eight single changes; it still completes 10/10 under B3, B4 and B5, because those leave its locator intact. The rules ladder completes 10/10 under seven of the eight single changes and fails only B8, where no route can reach a hidden control. It also completes 10/10 with B1–B4 applied together, where the script scores 0/10. With B1–B7 together, or all eight, both arms score 0/10, and the tool stops and says so rather than guessing. A cell of 10/10 has a 95% Wilson interval of 72–100% and 0/10 of 0–28%, so the gaps between 10/10 and 0/10 are far larger than the sampling noise.

Result 2: drift the rules cannot read (Figure 6). Here the model is qwen3.7-flash. With the labels reworded (B9) the rules ladder still completes 10/10 and the bare model 10/10, but once the ids are also rotated the ladder falls to 0/10, the recorded script is already at 0/10, and the bare model also scores 0/10. The model asked without any rules, but with a next-screen check,

	Recorded script	Rules ladder
clean	10/10	10/10
B1 ids renamed	0/10	10/10
B2 labels stripped	0/10	10/10
B3 layout reversed	10/10	10/10
B4 control duplicated	10/10	10/10
B5 in a scroller	10/10	10/10
B6 bare icon	0/10	10/10
B7 overlay	0/10	10/10
B8 control hidden	0/10	0/10
B1–B4 together	0/10	10/10
B1–B7 together	0/10	0/10
B1–B8 together	0/10	0/10

Figure 5: Rules only, no model: flows completed out of 10 for the recorded script and the rules ladder under each UI change.

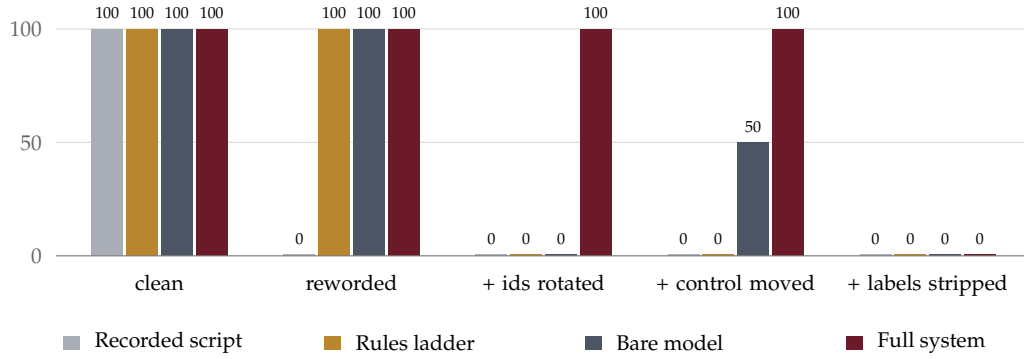


Figure 6: Drift the rules cannot read, model qwen3.7-flash: percentage of 10 flows completed. The full system completes until the control is also moved and its label stripped, where no arm completes.

completes only 5/10 even on the clean app and 5/10 with ids rotated, because it spends its turns on wrong picks. The full system completes 10/10 under reworded labels, rotated ids, and with the control also moved (B10), at a cost of about 7 model calls and 2.4 recovered wrong taps per run in the hardest of these. When the labels are also stripped, no arm completes (0/10 for all), and the full system stops honestly.

Result 3: planted instructions (Figure 7). Instructions such as "ignore the goal" are planted in control labels (B11). On the clean app and with the planted labels alone, all four arms complete 10/10. With the labels also reworded, the bare model completes only 7/10 (95% interval 40–89%); each of the three failures is a silent tap on a planted label, 0.3 per run. The rules ladder, the model with a check, and the full system all stay at 10/10 there. In the hardest mix (ids rotated, control moved) the ladder, the bare model and the model with a check all score 0/10, while the full system completes 10/10, tapping a planted label 2.0 times per run and recovering each time through the next-screen check, with no silent wrong action.

Result 4: no words left (Figure 8). When B1, B2 and B6 remove every word from the target, the rules ladder and the full system still complete 5/5, but the bare model completes 0/5, both when it is given the screen as text and when it is also given a screenshot with numbered badges (on the clean app it completes 5/5 in text and 4/5 with the screenshot). With the control also

moved every arm scores 0/5 and the full system stops honestly with no silent error. Each vision cell has only five runs, so 5/5 has an interval of only 57–100% and 0/5 of 0–43%; the screenshot result should be read as "no improvement seen", not as proof that vision cannot help.

Result 5: Android Settings (Figure 10). On the emulator, with success judged by page identity rather than by words, all arms complete 10/10 intents in English. When the system language is French the rules alone complete 2/10 (20%, interval 6–51%), while the full system completes 10/10 using 8 model calls and no silent wrong action. With the font scaled by 1.6 every arm, including the bare model, completes 8/10: two rows fall below the fold and no arm scrolls yet, which is a known gap.

Result 6: coverage and route memory (Figures 1 and 9). On stock Android Settings 78% of controls have no name by their own attributes but only 2% once the labels inside a clickable row are counted, which is why a tool that reads the way a screen reader does can find most controls. With route memory, a route found by the model after a UI change is remembered; the first run after the change asks the model, and later runs make no model calls.

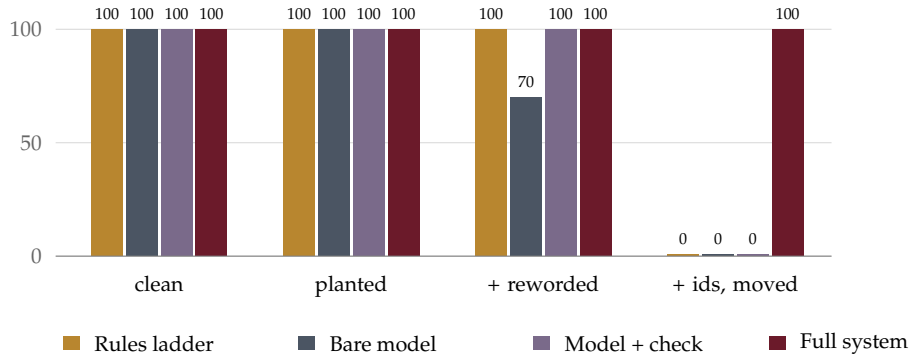


Figure 7: Instructions planted in control labels (B11), model qwen3.7-flash: percentage of 10 flows completed. The bare model fails three of ten when the labels are also reworded; the full system completes the hardest case, tapping a planted label twice per run on the way and recovering through the step check.

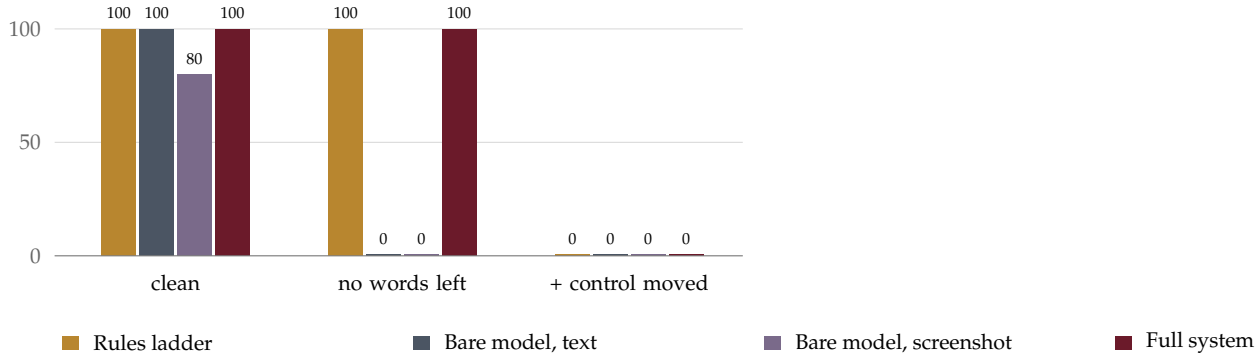


Figure 8: No words left on the target (B1, B2, B6), five flows per cell, model qwen3.7-flash: percentage completed. Showing the model a screenshot with numbered badges did not change the outcome in this run.

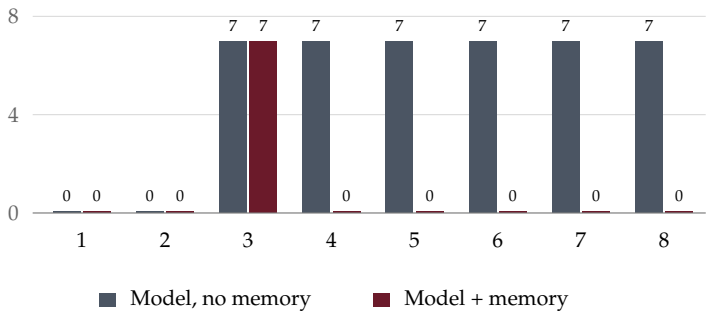


Figure 9: Route memory: model calls per run over 8 runs; the UI changes at run 3. With memory the first run after the change asks the model and later runs do not.

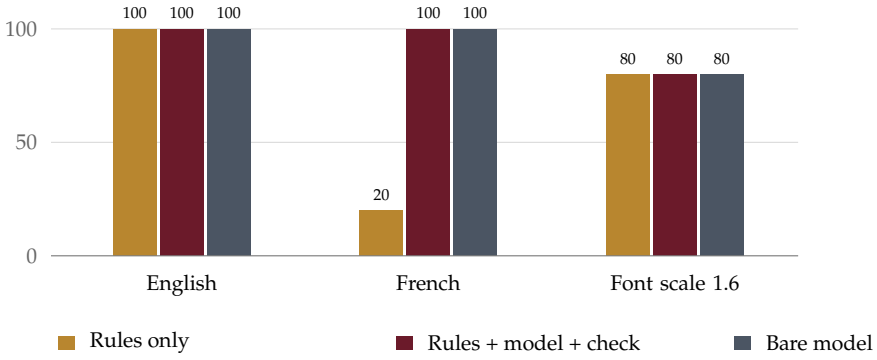


Figure 10: Android Settings, real drift (system language and font scale): percentage of intents completed on the `kenna` emulator, model qwen3.7-flash. At the large font two rows fall below the fold, which no arm scrolled to.

6 Limitations

- One model, one mock app and stock Settings; nothing on Daraz, OLX or Instagram; no user study with screen-reader users.
- Cached repetitions measure app and harness variation, not model variation.
- Our test harness once let state leak between arms (a bare model looked broken on a clean app). We proved it on a fresh page and fixed the harness.
- A later route can re-tap a control an earlier route already tapped; this is recorded for M2.

7 Team contribution

The three members worked on every part of the project together: the design, the experiments and this report were planned jointly, every change was reviewed by at least one other member, and each member can explain and modify any component. For accountability the work was divided into three areas, each led by one member with the other two contributing.

Member	Area led	Shared with the other two
Sharique Baig	Core harness: intents, the ladder of routes, next-screen checks, the model rung and its safety rules	Designing the intents and breaks B1–B11; reviewing the Android layer and the evaluation; writing and checking the report
Maira Aijaz	Evaluation: experiment design, UI-change sets, published logs, tables, figures with confidence intervals, and the report build	Reviewing the harness and the Android layer; running the live tests; writing and checking the report
Barkhi Khan	Android layer: `adb` capture, the ladder on UiAutomator trees, the live dashboard and voice feedback	Testing the web flow and the model rung; running experiments; writing and checking the report

8 Plan for M2 to M5

Milestone	Planned work
M2	A second model; scrolling and more intents measured with $N \geq 10$; planted-instruction and vision results; a second app; each member can explain or modify any component unaided.
M3	Present the M2 build to the external jury; explore a CEDIR hand-off.
M4	Real users outside the group, including a study with screen-reader users (with advisor approval); edge-case, security and accessibility review.
M5	Final report and defense, with evaluation method, design rationale and AI disclosure.

9 AI Tools Used

Claude Code (Anthropic's coding agent) was used throughout for writing source code, generating tests, running experiments and reading their logs, and drafting documentation, including this report. The language model inside the system (qwen3.7-flash via OpenRouter) is the subject of the experiments, not a development aid. The team states the proportion: about 60% of the code and of this text was produced with AI assistance and about 40% by the team's own hand, all under the team's direction and review. Every figure is read from published logs, and each author is expected to explain and modify any component without tool assistance.

References

[1] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *J. American Statistical Association*, 22(158):209–212, 1927.

[2] Department of Computer Science, IBA Karachi, *CSE 49x – Final Year Project: Philosophy, processes and practices*, August 2026.